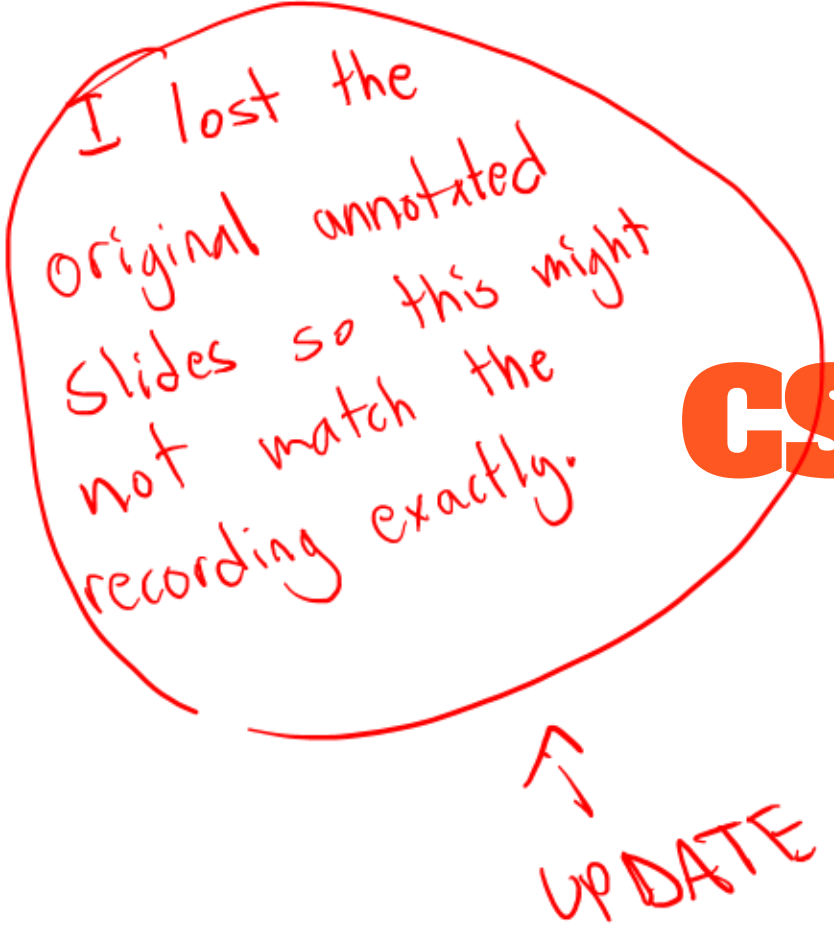


CS 340

HTTP

clicker.cs.illinois.edu

Q1

~Code~

101

Updates

1. Exam 2 Retake Graded

1. MP 6 is due today

1. MP 7 - aiohttp lite, due next week

1. HW 7 due Wednesday

CS 462 Issues of Law & Policy in Computer Science

- A CS technical elective covering the intersection of law and technology
- Read and discuss real-world legal cases
- Apply your technical knowledge to legal questions and controversies
- Learn about the laws and regulations governing computing technology
- Sample topics: hacking crimes, crypto scams, AI copyright infringement, NSA surveillance, code as protected speech, and digital ownership
- Scan this QR code for the full syllabus and course description

HTTP

Today's LGs - Build your understanding of an internet packet to include HTTP.

Be able to describe each of the three layers of an internet packet and what they accomplish
Be able to describe the format of HTTP
Be able to complete MP 7

- Python Decorators
- Python Async

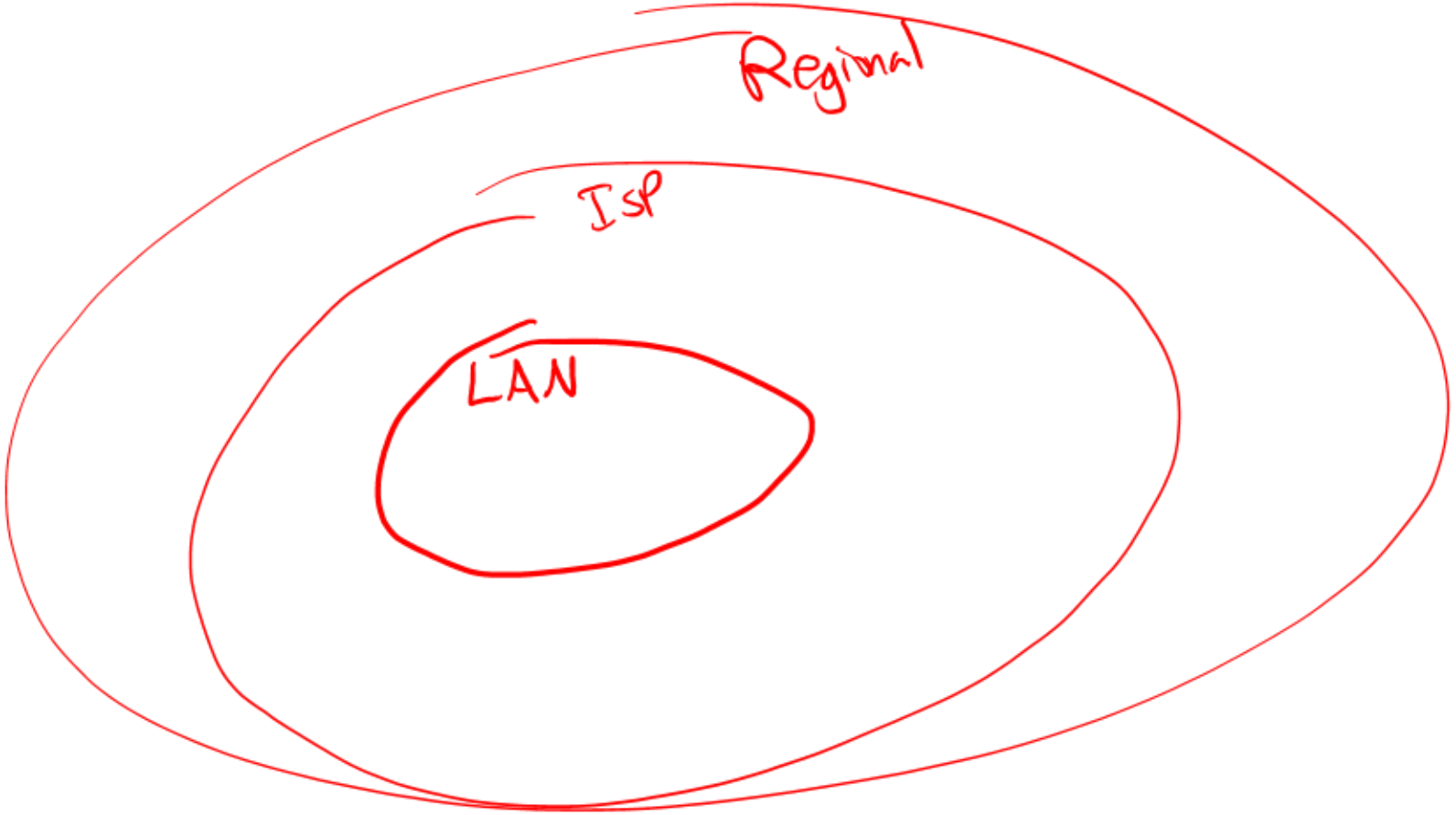
Agenda

1. Review

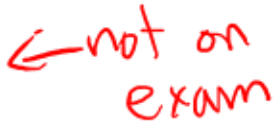
1. HTTP

1. Advanced Python

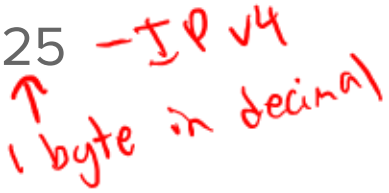
Internet Review



IP Addresses



192.168.1.25



2001:0db8:85a3:0000:0000:8a2e:0370:7334



How do VPNs work? → not on exam

if curious see lecture recording or slide notes

Which are True?

clicker.cs.illinois.edu

Q2

~Code~
340

A) The internet is a system of computers.

B) Packets can be any size.

C) IP addresses specifies which computer a packet needs to go to.

D) TCP is slower but more reliable than UDP

E) My LAN has a private IP address only I know.

public

Devices within your LAN have a private IP address

Layers of a packet

Layer	Protocols	Use
Application	HTTP, SSH, SMTP, etc.	programmer-visible messages
Transport	**TCP**, **UDP**	reach a specific program
Internet	**IP**, which has two versions: IPv4 and IPv6	reach a specific computer

Application Layer Protocol

Purpose	Protocol	Transport
Web	HTTP	TCP
Web (encrypted)	HTTPS	TCP, or UDP via QUIC (HTTP/3)
Email	SMTP (send), IMAP and POP3 (retrieve)	TCP
Remote login	SSH	TCP
Name lookups	DNS	UDP (TCP for large replies)

HTTP – hyper text transfer protocol – uses client server model

client – requests things

server – responds to requests

HTTP – specifies the format of the requests and responses

sits below IP and TCP ← needs these to get the packet to the correct computer

HTTP

Request Methods

GET - asks for a resource

POST - sends data to the server

PUT - sends data to the server (overwrites existing)

DELETE - removes a resource

Responses

200 - ok

400 - bad request

404 - not found

500 - internal Server Error

HTTP - Format Example 0b01

Request

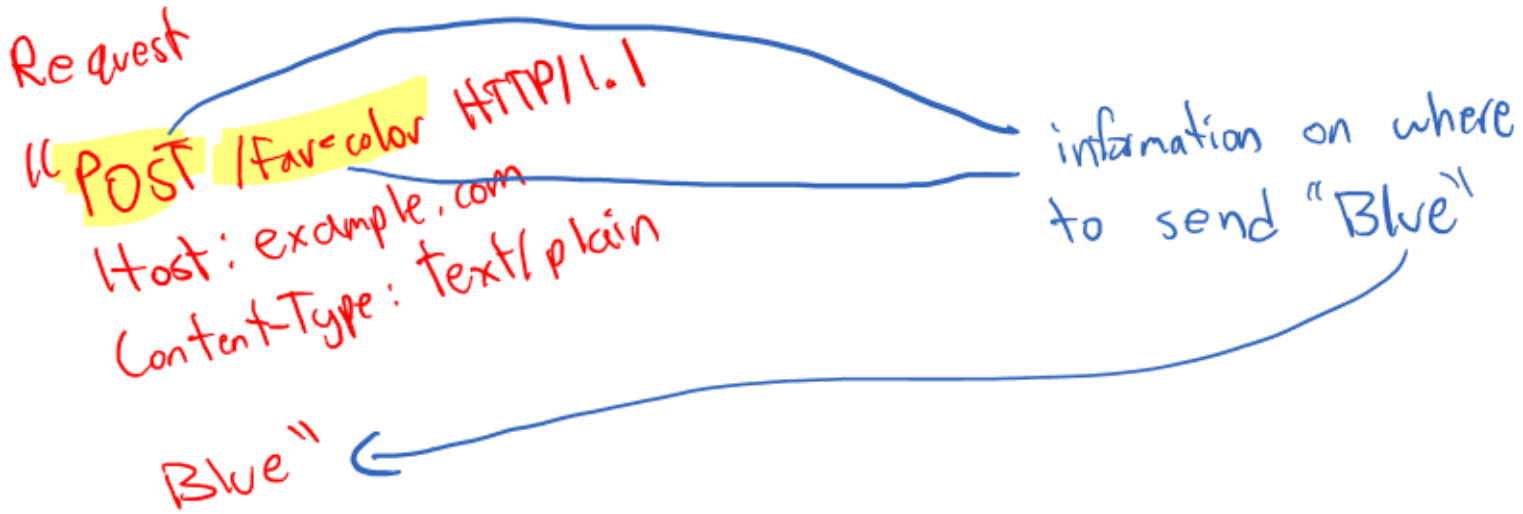
"GET /index.html HTTP/1.1 ← bytes encoding these characters are what is sent.
Host: example.com"

Response

"HTTP/1.1 200 OK
Content-Type: text/html

<html>Hello!</html>"

HTTP - Format Example 0b10



With a partner: Why do you think we use HTTP with TCP and not UDP?

We need reliability instead of speed. We don't want a long http request or response to be lost and not resent.

What internal numbering system is used to get the HTTP info to the right process?

clicker.cs.illinois.edu

Q3

~Code~
101



Can a client send an HTTP request to a server physically located somewhere else without the IP layer?

clicker.cs.illinois.edu

Q4

~Code~
101

No! we need the IP layer

What gets an HTTP request to the correct server function once the packet is at the correct process?

clicker.cs.illinois.edu

Q5

~Code~
101



url + request method

Aiohttp - Python library that helps you write a web server

MP 7 - aiohttp Lite

After the IP and TCP layers are removed

1. Read in bytes from an incoming packet
2. Interpret the bytes to understand the incoming request
3. Funnel the request body to the corresponding function
    a. The function will call your response constructor to create a response
4. Send that response out

Agenda

1. Review Clicker

1. HTTP

1. **Advanced/Specific Python**
    a. **JSON**
    b. **Bytes**
    c. **Function decorators**
    d. **Async**

JSON

```
{
  "name": "Jule",
  "favorites": ["CS", "coffee"],
  "info": {
    "year": 2026
  }
}
```

← nested dictionaries and lists

Bytes in Python

```
tmp = "hello"
```
← complicated string type

```
tmp = tmp.encode("utf-8")
```
← tmp is now simple bytes representing the characters

```
tmp = tmp.decode("utf-8")
```
↑ reverse

Are encode() and decode() in-place? Or do they return a copy?

clicker.cs.illinois.edu

Q6

~Code~
101



Bytes in Python Example

```
2  var = "start"
3  x = [1, 4, 6]
4  for value in x:
5      var += " " + str(value)
6  print(var)
7  bytes_version = var.encode("utf-8")
8  print(bytes_version)
```



```
start 1 4 6
b'start 1 4 6'
```

Classes Example

```
1   class Book:
2       rating : int  = 0
3       def __init__(self, title: str, length: int):
4           self.title = title
5           self.length = length
6
7       def add_rating(self, rating: int):
8           self.rating = rating
9
10      def should_read(self):
11          if self.rating > 5 and self.length > 100:
12              return "yes"
13          else:
14              return "should not read"
15
16  bk = Book("from C to shining C++", 400)
```

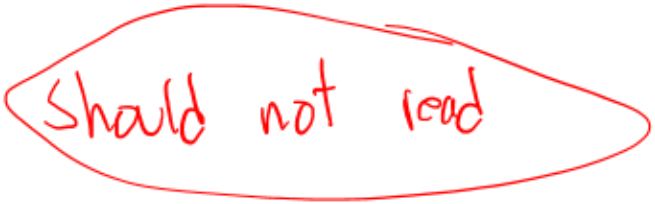
Classes, what would print?

```
1   class Book:
2       rating : int  = 0
3       def __init__(self, title: str, length: int):
4           self.title = title
5           self.length = length
6
7       def add_rating(self, rating: int):
8           self.rating = rating
9
10      def should_read(self):
11          if self.rating > 5 and self.length > 100:
12              return "yes"
13          else:
14              return "should not read"
15
16  bk = Book("from C to shining C++", 400)
17  print(bk.should_read())
```

clicker.cs.illinois.edu

Q7

~Code~
101



Classes + Bytes

```python
class Book:
    rating : int  = 0
    def __init__(self, title: str, length: int):
        self.title = title
        self.length = length

    def __bytes__(self):
        out = self.title + ": " + str(self.length)
        return out.encode('utf-8')

bk = Book("from C to shining C++", 400)
print(bytes(bk))
```

overwrite the bytes function

PROBLEMS OUTPUT TERMINAL 3

TERMINAL

```
root@3cb3343e38be:/workspaces/mp7# python3 class.py
b'from C to shining C++: 400'
root@3cb3343e38be:/workspaces/mp7#
```

Agenda

1. Review Clicker

1. HTTP

1. **Advanced/Specific Python**
    a. **JSON**
    b. **Bytes**
    c. **Function decorators**
    d. **Aysnc**

Functions - first-class objects

```
def greet():
    print("hi")

f = greet
f()
```

Functions - first-class objects

```
def greet():
    print("hi")
```

```
def run(func):
    func()

run(greet)
```

Functions - first-class objects

```
def make_greeter(name):
    def greet():
        print(f"Hello {name}!")
    return greet

greeter = make_greeter("Jule")
greeter()
```

What goes in the black box?

```
routes = {}

def register(path, func):
    routes[path] = func

def hello():
    print("Hello!")

register("/hello", [redacted])

routes["/hello"]()
```



clicker.cs.illinois.edu

Q8

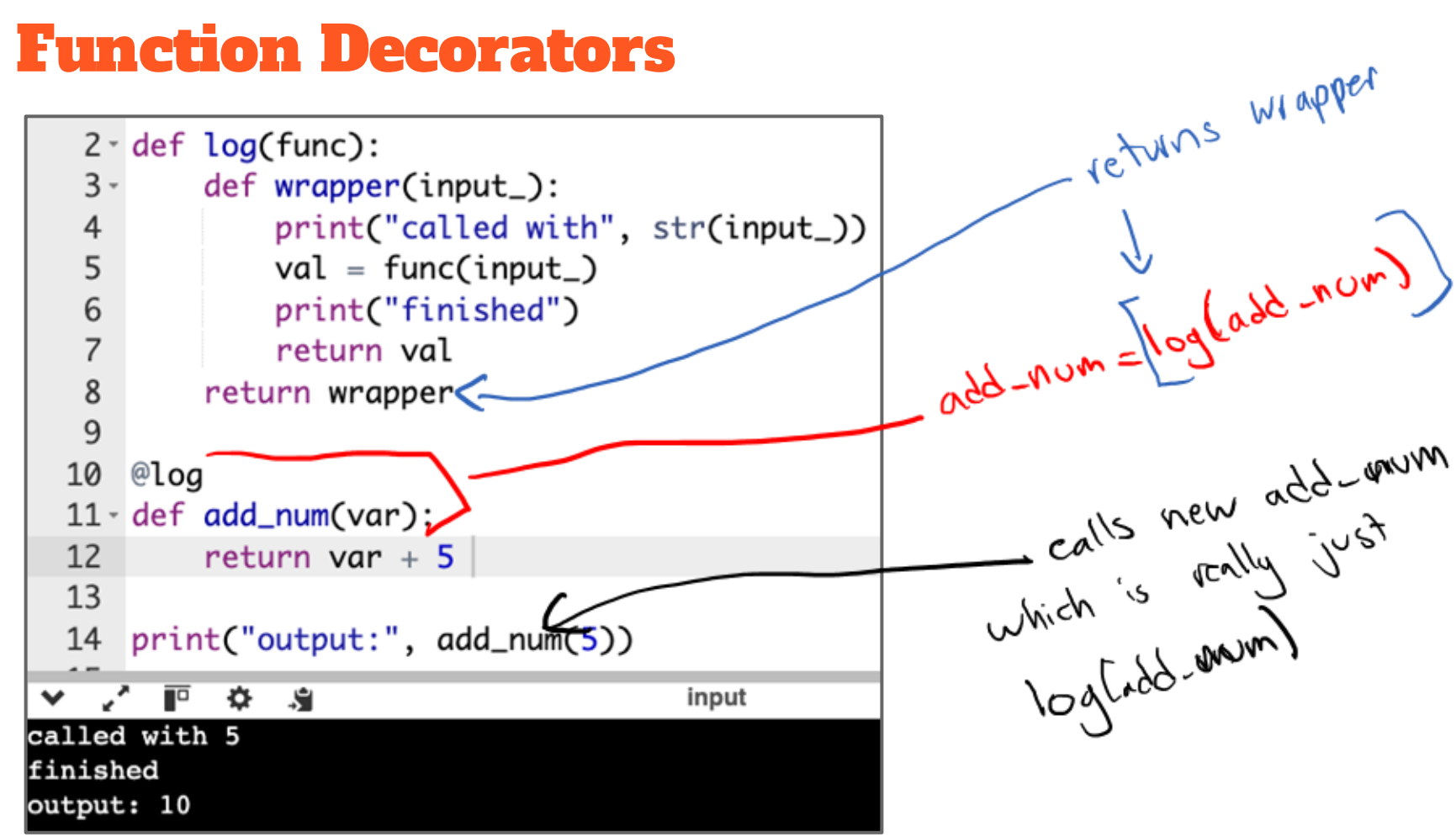
~Code~
101

Function Decorators

```
2  def log(func):
3      def wrapper(input_):
4          print("called with", str(input_))
5          val = func(input_)
6          print("finished")
7          return val
8      return wrapper
9
10 @log
11 def add_num(var):
12     return var + 5
13
14 print("output:", add_num(5))
```

input

```
called with 5
finished
output: 10
```



What prints?

```
2  def call_twice(func):
3      def wrapper(in_):
4          func(in_)
5          func(in_)
6      return wrapper
7
8  @call_twice
9  def print_names(names):
10     for i in names:
11         print(i)
12
13 names_ = ["jule", "luther"]
14 print_names(names_)
```

clicker.cs.illinois.edu

Q9

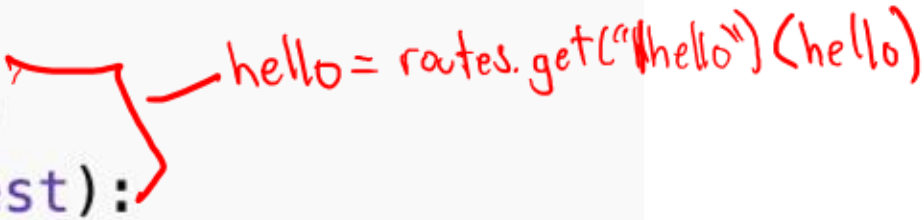
~Code~
101



Function Decorators

```
routes = web.RouteTableDef()

@routes.get("/hello")
async def hello(request):
    return web.Response(text="Hello!")
```



```python
routes = web.RouteTableDef()

@routes.get("/hello")
async def hello(request):
    return web.Response(text="Hello!")
```

Server

aiohttp library code

```python
class RouteTableDef:
    def __init__(self):
        self.routes = {}

    def get(self, path):
        def decorator(func):
            self.routes[("GET", path)] = func
            return func
        return decorator
```

Aiohttp function decorators

server code

```
60    @routes.get("/hint")
61    async def give_game_info(request: web.Request) -> web.Response:
62        return web.Response(text="no hints!")
63
64
65    if __name__ == '__main__':
66        app = web.Application()
67        app.add_routes(routes)
68        web.run_app(app) # this function never returns
```

Agenda

1. Review Clicker

1. HTTP

1. **Advanced/Specific Python**
    a. **Json**
    b. **Bytes**
    c. **Function decorators**
    d. **Async**

Async

Python async - let's a program switch between coroutines while one is waiting for something slow (waiting for messages over the internet)

coroutine - special function that can pause and resume

Async

```
 2  import asyncio
 3
 4  async def task1():
 5      print("task1 start")
 6      await asyncio.sleep(1)
 7      print("task1 done")
 8
 9  async def task2():
10      print("task2 start")
11      await asyncio.sleep(2)
12      print("task2 done")
13
14  async def main():
15      await asyncio.gather(task1(), task2())
16
17  asyncio.run(main())
```

switches at await keyword

```
task1 start
task2 start
task1 done
task2 done
```

Async

Idea - async let's the program switch between coroutines while one is waiting for something slow (waiting for messages over the internet)

- Use the await keyword to indicate the program can switch if its waiting.
- await can be used to call another async function

Async Aiohttp

```
from aiohttp import web

routes = web.RouteTableDef()

@routes.post("/echo")
async def echo(request):
    data = await request.text()
    return web.Response(text=data)

app = web.Application()
app.add_routes(routes)

web.run_app(app)
```

Compared to threads
small overhead

MP 7 - aiohttp Lite

1. Read in bytes from an incoming packet
2. Interpret the bytes to understand the incoming request
3. Funnel the request body to the corresponding function
    a. The function will call your response constructor to create a response
4. Send that response out